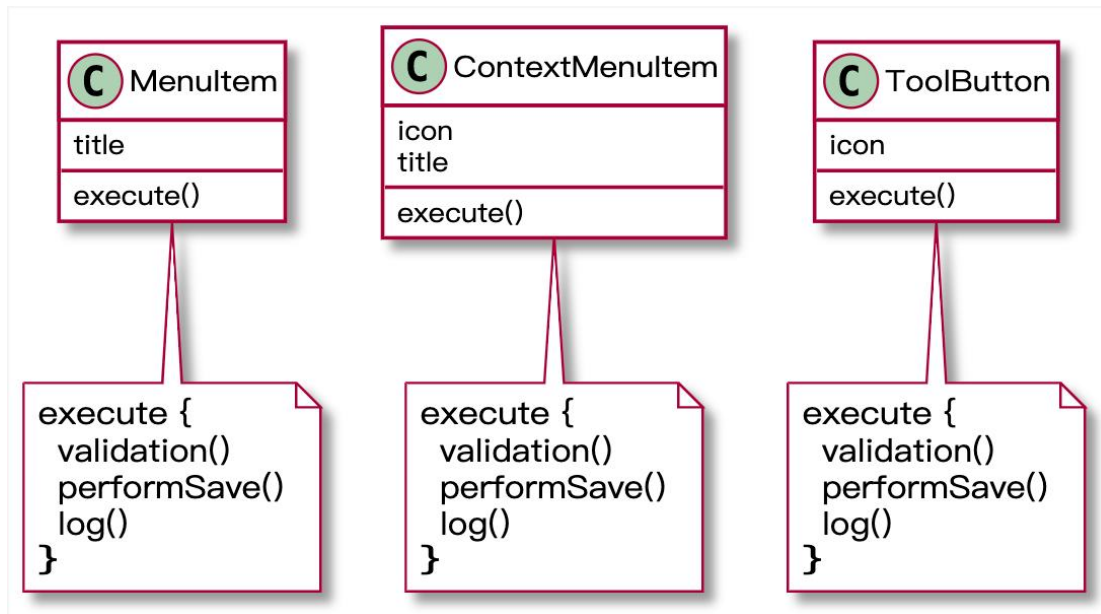


Command



场景1： 通过菜单等入口执行命令

- 在一个UI应用中，用户可以通过主菜单(MenuItem)，鼠标右键(ContextMenuItem)，工具条(ToolButton)等方式执行一个保存文件的操作。可以设计如下：



场景：通过菜单等入口执行命令

- 该设计方案的问题：
 - 保存文件的代码有重复
 - 保存文件的业务逻辑混杂在UI的业务逻辑之中
 - 无法进行undo/redo的操作



场景：通过菜单等入口执行命令

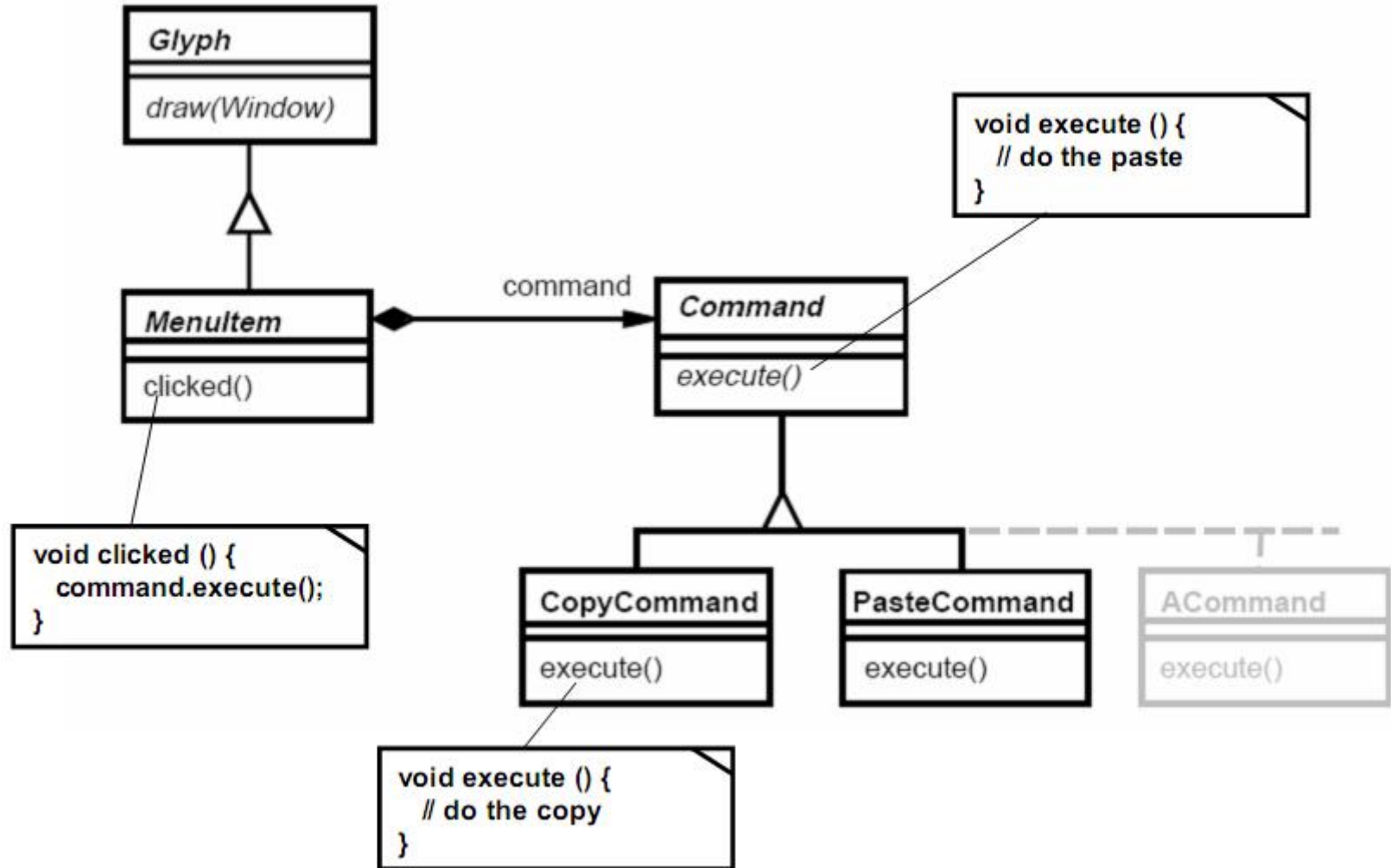
- 围绕命令的相关需求：
 - 可能需要以不同的方式呈现：按钮，菜单或者命令行
 - 可能需要进行权限的管理，根据不同的用户呈现为enable/disable的形式
 - 可能需要支持undo/redo
 - 可能需要支持宏记录
 - 可能需要支持离线模式

Motivation

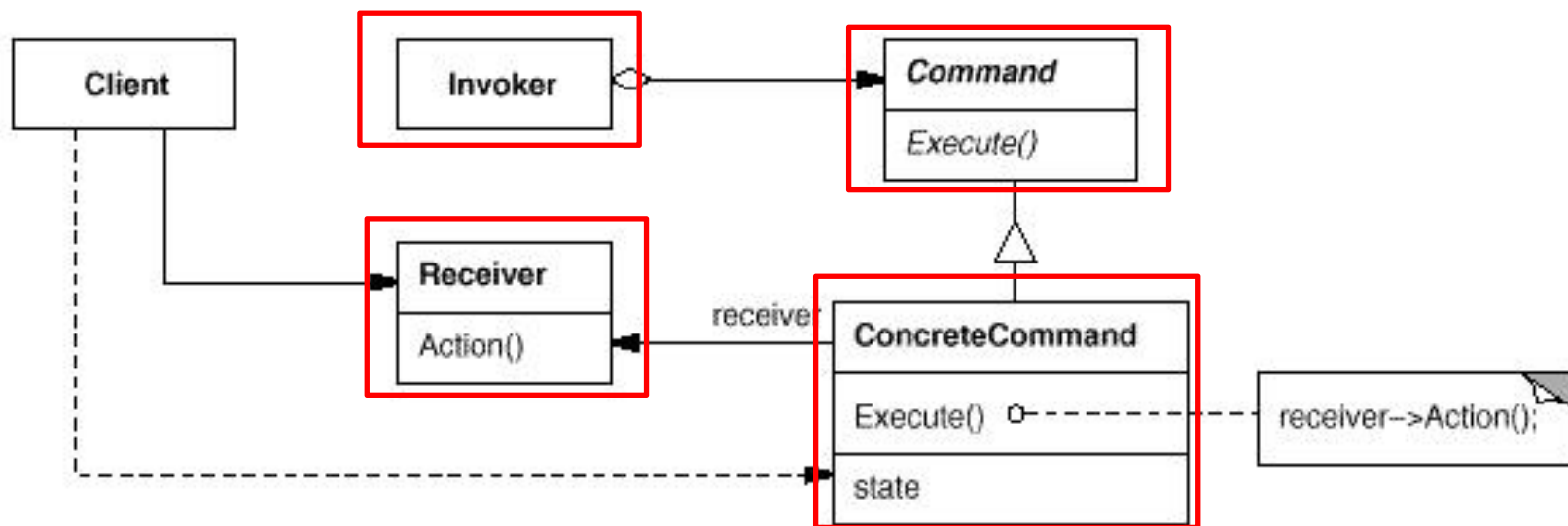
- Sometimes it's necessary to issue requests to objects without knowing anything about the operation being requested.
- Support redo/undo history operation
- Support synchronize operation across network
- Support Macro/Replay operations



Sample: User Operations



Structure

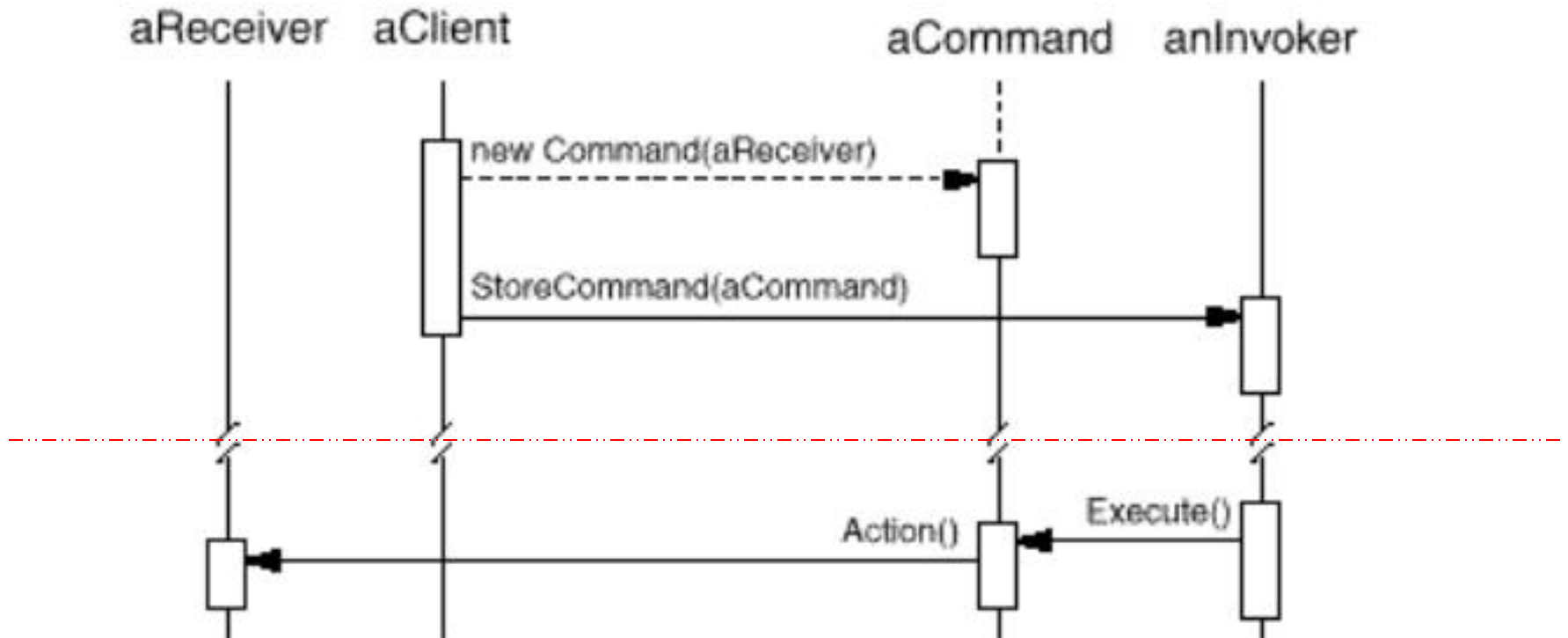


Participants

- Command
 - Declared an interface for executing an operation.
- ConcreteCommand
 - Defines a binding between a Receiver object and an action
 - Implements Execute by invoking the corresponding operation(s) on Receiver
- Client
 - Create a ConcreteCommand object and sets its receiver
- Invoker
 - Asks the command to carry out the request
- Receiver
 - Knows how to perform the operations associated with carrying out a request. Any class may serve as a receiver

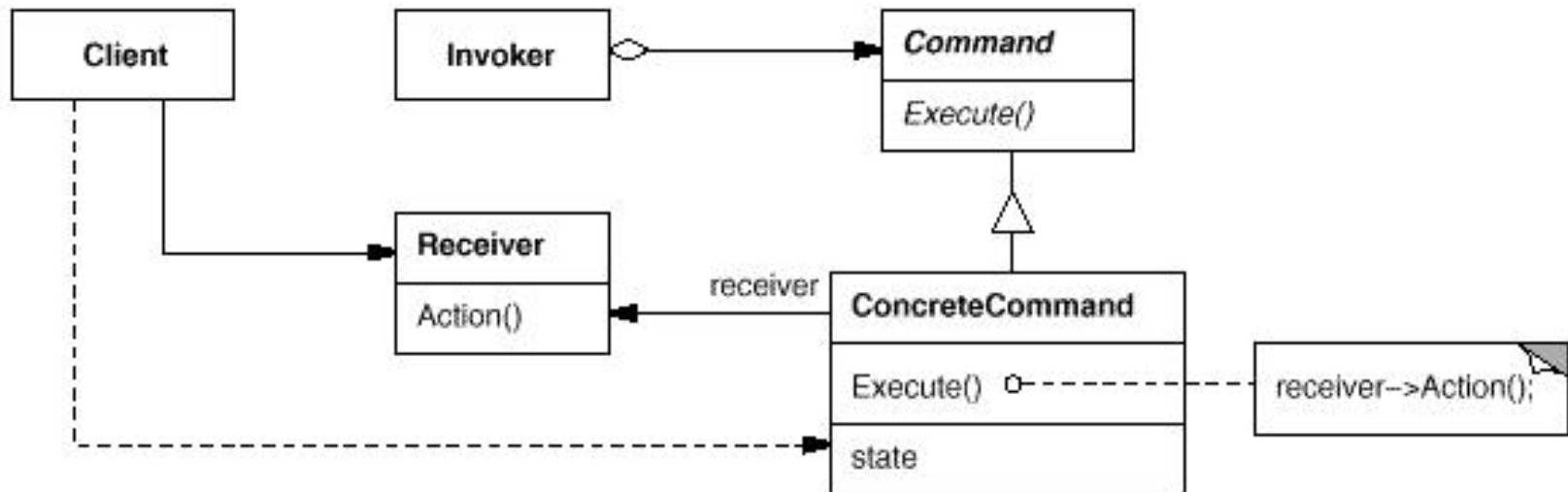


Collaboration



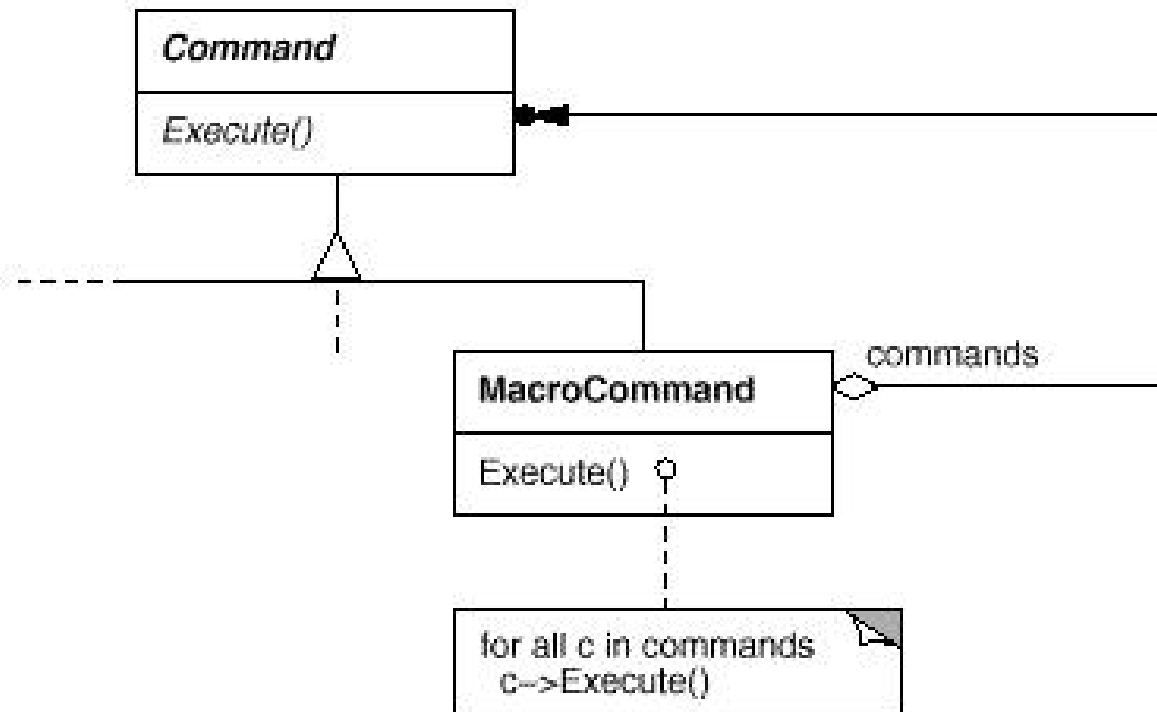
Consequences

- Command decouples the object that invokes the operation from the one that knows how to perform it.



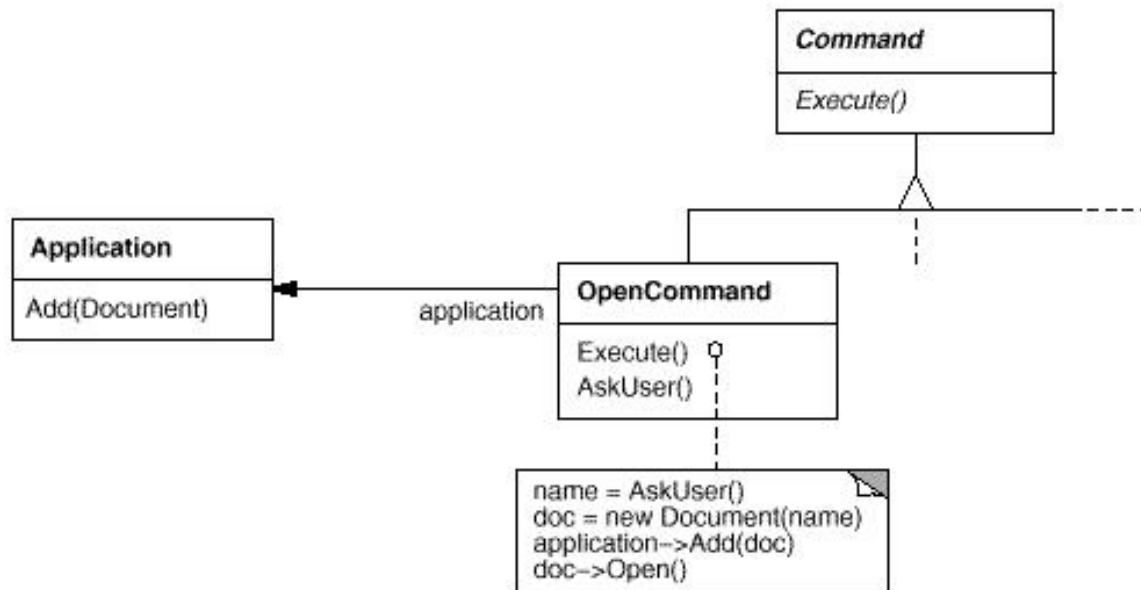
Consequences(Cont.)

- You can assemble commands into a composite command.



Consequences(Cont.)

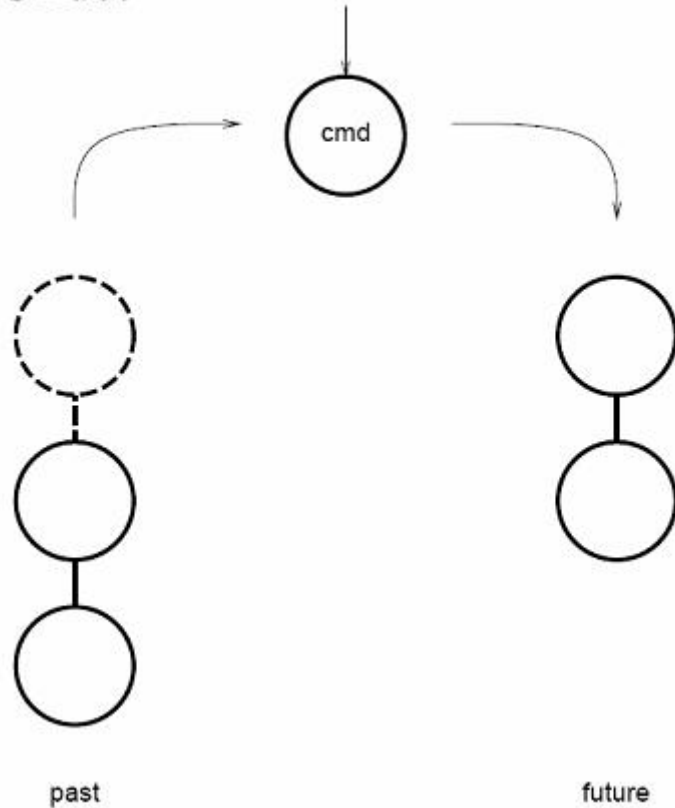
- It's easy to add new Commands, because you don't have to change existing classes.



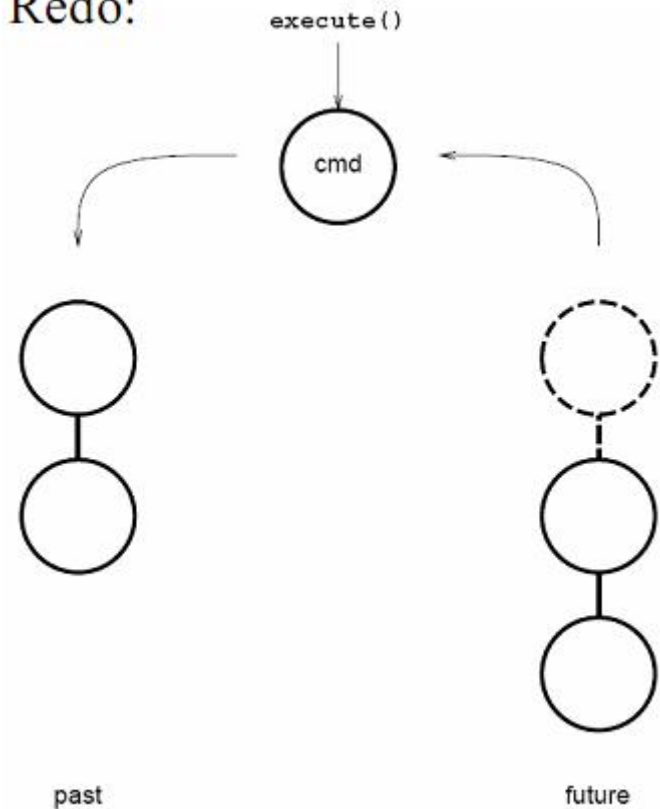
Consequences(Cont.)

- List of commands defines execution history
- Command Stack

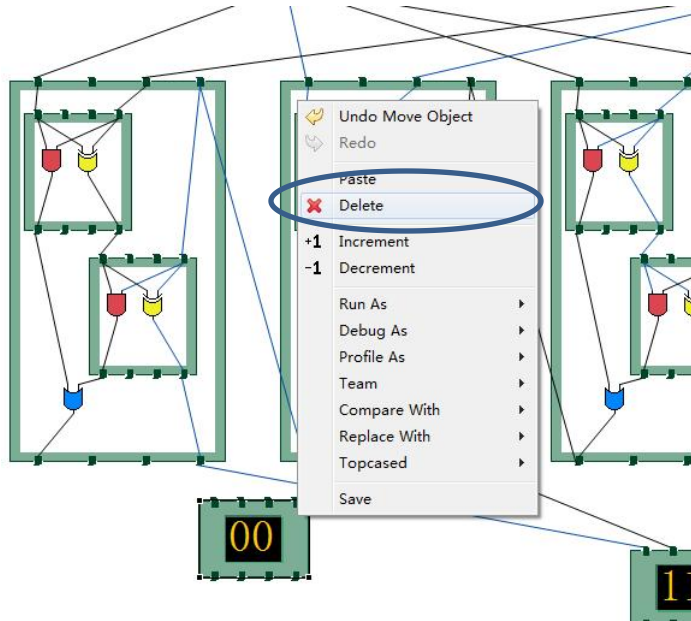
Undo:



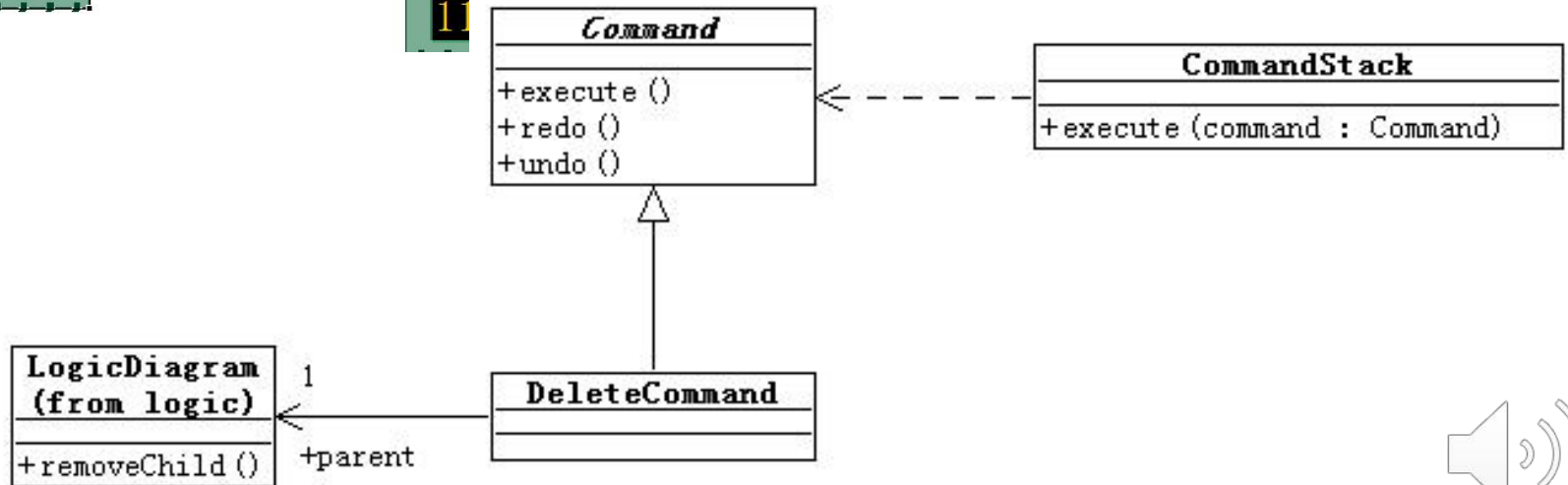
Redo:



GEF: Diagram Command



```
Class DeleteCommand extends Command{  
    public void execute() {  
        deleteConnections(child);  
        detachFromGuides(child);  
        index =  
        parent.getChildren().indexOf(child);  
        parent.removeChild(child);  
    }  
}
```



Implementation

- How intelligent should a command be?
 - Merely defines a binding between a receiver and the actions that carry out the request
 - Or implements everything itself without delegating to a receiver at all
- Supporting undo and redo
 - Need a history management mechanism.

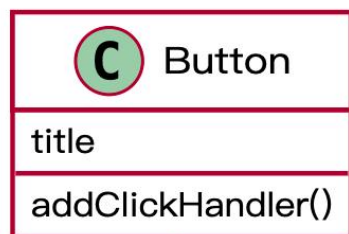


思考题

- 如果我们需要一个按钮，按下按钮后执行保存当前文件的动作。
- 下面从简单到复杂一共有**5**种方式，请大家思考一下，在什么情况下，采用什么方式。



方式1



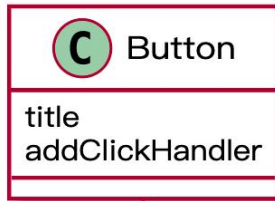
直接在addClickHandler中写处理逻辑

```
class Button{
    constructor(readonly title : string){}
    addClickHandler(handler:()=>void){
        //...
    }
}

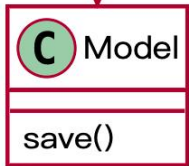
const button1 = new Button('save')
button1.addClickHandler(()=>{
    //save logic:
    // validation
    // perform save
    // log
})
```



方式2



业务逻辑不写在addClickHandler中，
而是写在一个独立的类：Model中

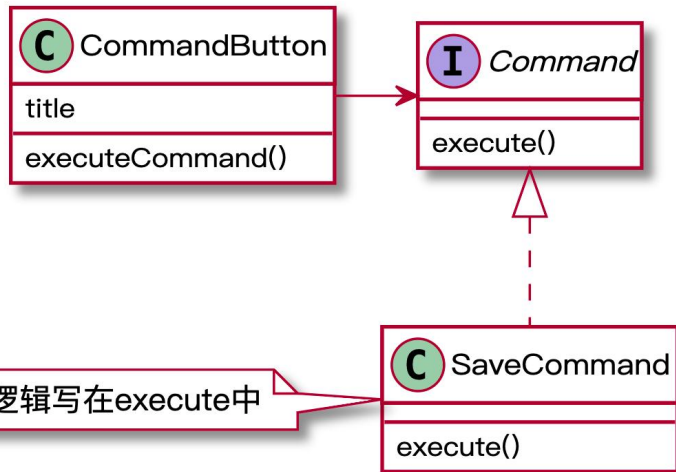


```
class Model{
    save(){
        //check logic
        //perform delete logic
        //log logic
    }
}

const model = new Model();
const button2 = new Button('save');
button2.addClickHandler(()=>{
    model.save()
})
```



方式3



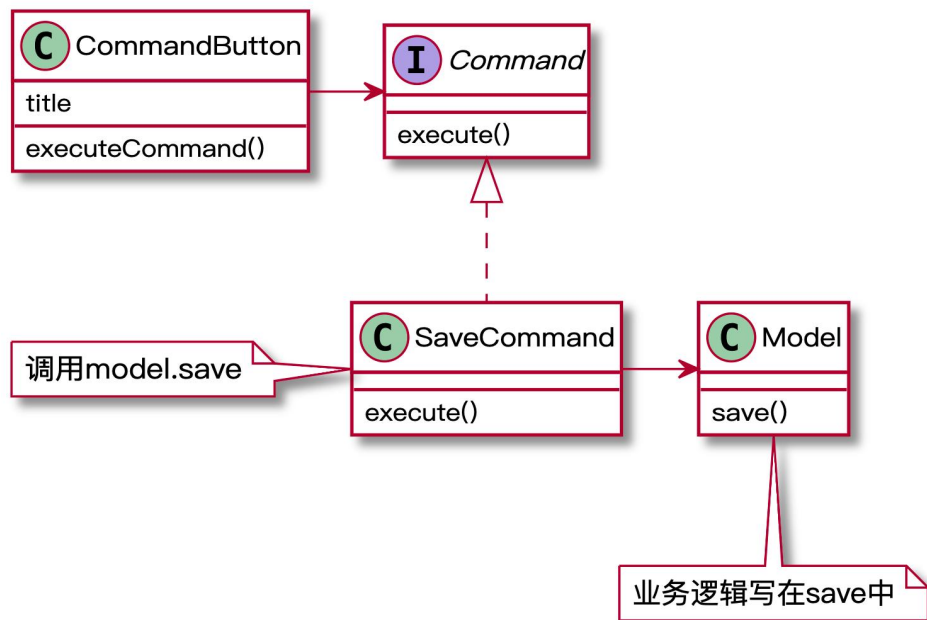
```
interface Command{
    execute():void;
}

class SaveCommand implements Command{
    execute():void{
        //check logic
        //perform delete logic
        //log logic
    }
}

class CommandButton{
    constructor(
        readonly title:string,
        readonly command:Command
    ){
    }
    executeCommand(){
        this.command.execute()
    }
}
```



方式4



```
class Model{
    save(){
        //check logic
        //perform delete logic
        //log logic
    }
}

const model = new Model();

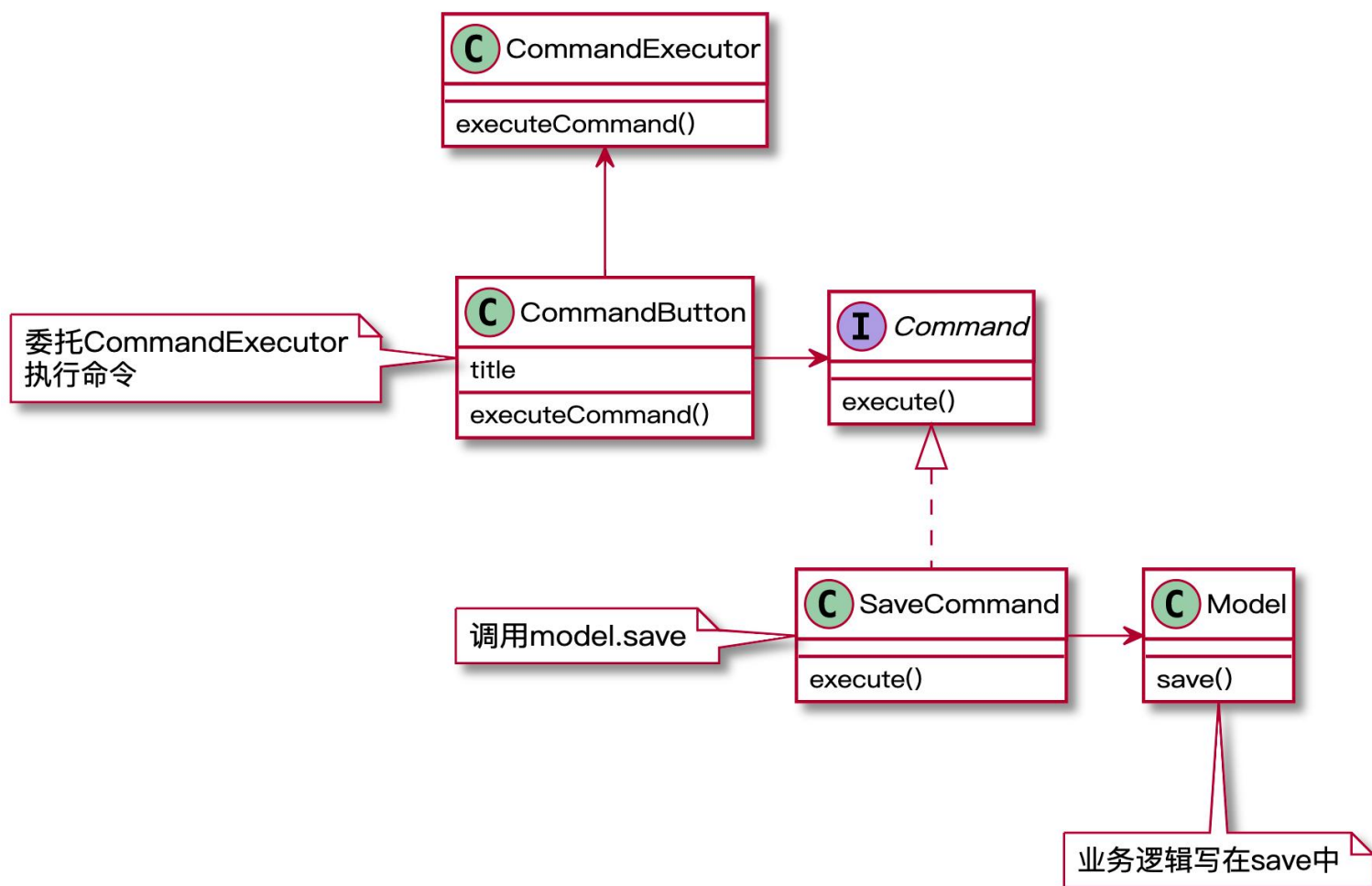
interface Command{
    execute():void;
}

class SaveCommand implements Command{
    execute():void{
        model.save()
    }
}

class CommandButton{
    constructor(
        readonly title:string,
        readonly command:Command
    ){
    }
    executeCommand(){
        this.command.execute()
    }
}
```



方式5



方式5

```
class Model{
    save(){
        //check logic
        //perform delete logic
        //log logic
    }
}

const model = new Model();

class CommandExecutor{
    executeCommand()
}

const commandExecutor : CommandExecutor
```

```
interface Command{
    execute():void;
}

class SaveCommand implements Command{
    execute():void{
        //check logic
        //perform delete logic
        //log logic
    }
}

class CommandButton{
    constructor(
        readonly title:string,
        readonly command:Command
    ){
    }
    executeCommand(){
        this.command.execute()
    }
}
```

